

Decisiones de Arquitectura

Cómo está organizado el sistema y por qué se hizo así

Para quién es este documento. El equipo técnico que desarrolla, despliega o mantiene GeopetroIO. Resume las decisiones que dan forma al sistema y el motivo de cada una.

Cómo leerlo. La parte 1 muestra el conjunto y el flujo de datos. La parte 2 presenta cada decisión en el formato contexto, decisión y consecuencias. La parte 3 lista los riesgos aceptados y los pendientes. Los detalles están en las specs del repositorio, indicadas en cada decisión.

Situación. En octubre de 2026 el sistema todavía no está en producción. Por eso algunos cambios entraron sin migración gradual ni compatibilidad con nombres anteriores.

Nota sobre los nombres. Las pantallas y el código están en portugués. Cuando aparece un nombre propio del sistema, se mantiene como está, por ejemplo *card* (tarjeta) o *Unidade* (unidad).

Contenido

1. Visión general	3
Flujo de arquitectura	4
Dentro del Desktop	5
2. Decisiones	6
D01 · Monorepo con aplicaciones independientes	6
D02 · Braserv-Core es dueño de la identidad y del registro organizacional	6
D03 · Solo el Core emite tokens, firmados con RS256	6
D04 · Un solo origen a través del proxy	7
D05 · Dos caminos independientes para la telemetría	7
D06 · MQTT por lotes con mensaje autodescriptivo	7
D07 · Servicio de Telemetría con InfluxDB	7
D08 · Cómo se identifica la unidad	8
D09 · Desktop como agente de borde dirigido por configuración	8
D10 · Identidad estable de la tarjeta	8
D11 · La calibración se queda en la estación	8
D12 · El servidor registra la alarma y la estación la señala	9
D13 · Configuración restringida, con sesión solo en memoria	9
D14 · Despliegue separado por camino de datos	9
D15 · Arquitectura hexagonal por módulo	9
3. Riesgos aceptados y pendientes	10
Dónde profundizar	10

1. Visión general

GeopetroIO recoge datos de unidades en campo, como taladros, unidades de bombeo y cementación, y los entrega a la supervisión remota y al cliente. Son seis aplicaciones independientes.

Aplicación	Tecnología	Función
Braserv-Core	Spring Boot · Java 21	Identidad, inicio de sesión, tokens y registro organizacional: usuarios, empresas, regionales, sectores y unidades.
Geopetro-Backend	Spring Boot · Java 21	Monitoreo, tiempo real, alarmas, configuración de tarjetas y simulador de cementación.
Geopetro-Front	Angular 21 · Taiga UI	Aplicación web para registros, monitoreo, alarmas, simuladores e informes.
Geopetro-Desktop	JavaFX 21 · Spring Boot	Agente de borde en el PC de la unidad. Lee el PLC LOGO!, muestra el panel local y publica la telemetría.
Geopetro-Telemetry	Spring Boot · InfluxDB	Consume MQTT, guarda las series en InfluxDB y responde las consultas de histórico al Backend.
Braserv-Horus	JavaFX 21 · Gradle	Aplicación de cementación autónoma. Lee el PLC de la bomba y genera la carta de operación en PDF.

Flujo de arquitectura

El diagrama muestra el despliegue en dos máquinas virtuales. Los números de las flechas corresponden a la tabla de abajo.

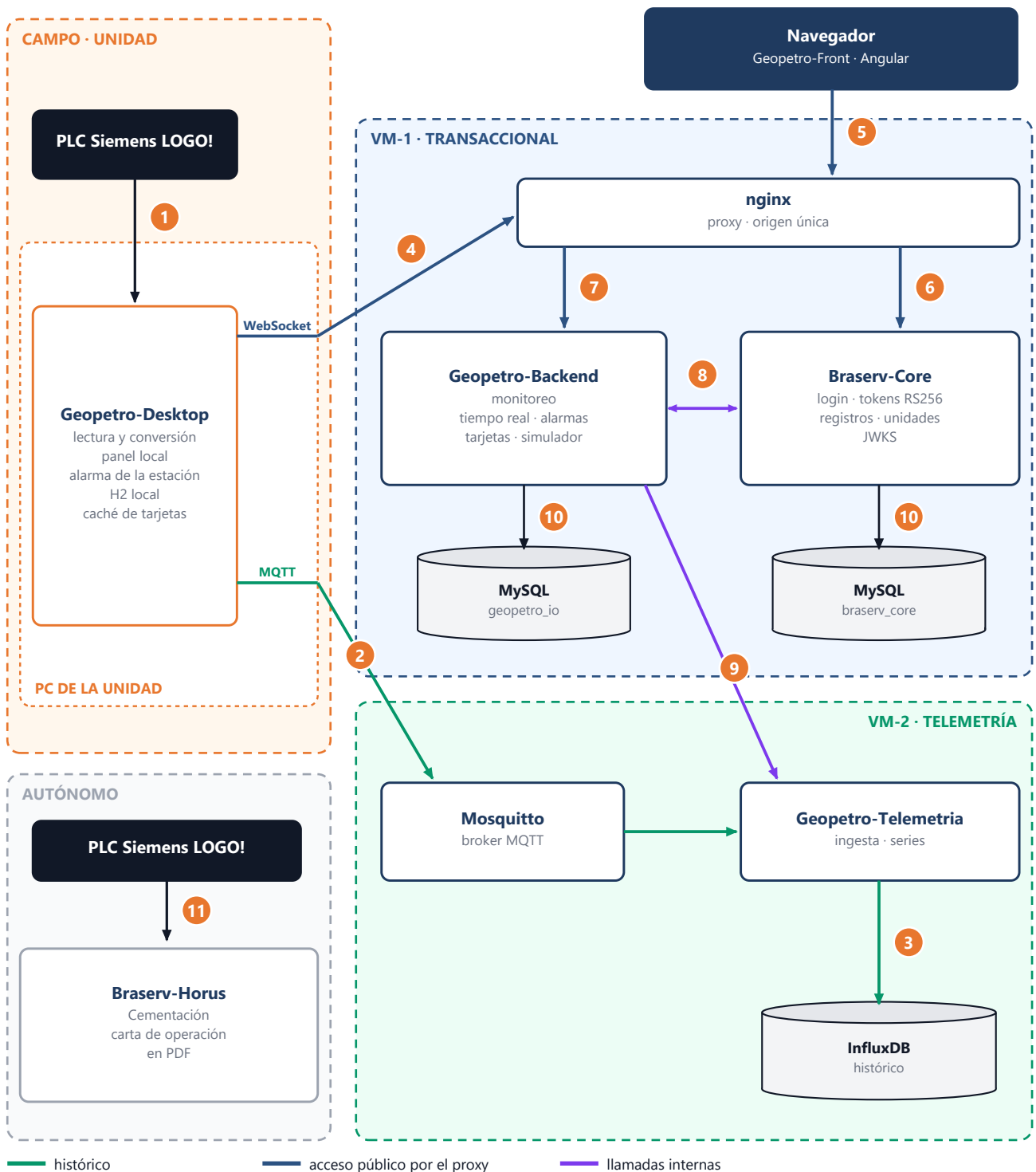


Figura 1 — Flujo de arquitectura de GeopetroIO.

N.º	Origen y destino	Protocolo	Para qué
1	PLC LOGO! → PC de la unidad	S7 · TCP 102	Geopetro-Desktop, en el PC de la unidad, lee la memoria V del LOGO! en cada ciclo, normalmente 1 s, y convierte cada valor según el tipo de tarjeta.
2	PC → Mosquitto	MQTT · QoS 1	Histórico. Un mensaje por ciclo en el tópico telemetria/{idUnidade}/batch.
3	Telemetría → InfluxDB	HTTP · lote	Escritura asíncrona en lotes de 500 puntos o 1 s, en UTC.
4	PC → nginx	HTTPS · WebSocket	Inicio de sesión en el Core, envío del tiempo real por STOMP y recepción del documento de tarjetas.
5	Navegador → nginx	HTTPS · WSS	Pantallas web y suscripción al tiempo real.
6	nginx → Core	HTTP	Inicio de sesión, usuarios, empresas, regionales, sectores, unidades y clientes de servicio.
7	nginx → Backend	HTTP · STOMP	Monitoreo, alarmas, simulador y /ws.
8	Backend ↔ Core	HTTP interno	Llaves JWKS, acceso actual del usuario, catálogo de unidades y vínculos antes de eliminar.
9	Backend → Telemetría	HTTP interno	Consulta de series históricas. El Front nunca llama a la Telemetría.
10	Core y Backend → MySQL	JDBC	Mismo servidor MySQL, bases de datos y usuarios separados.
11	PLC LOGO! → Horus	S7 · TCP 102	Cementación autónoma, sin integración con las demás aplicaciones.

Dentro del Desktop

Un hilo virtual lee el LOGO!, convierte los valores y entrega la lectura a cuatro destinos. Ningún destino bloquea la lectura ni a los demás destinos.

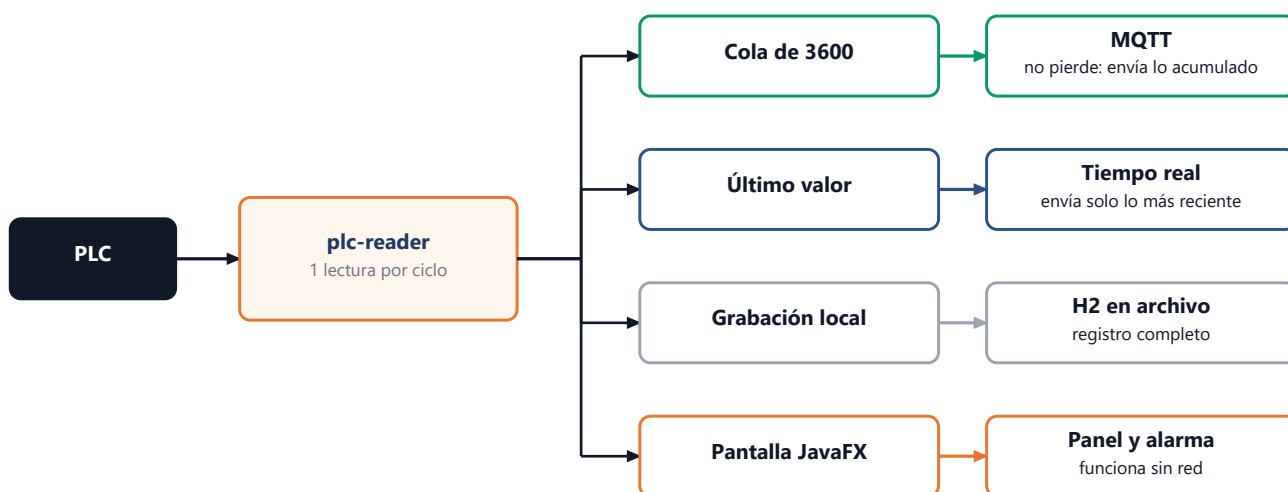


Figura 2 — Cómo se reparte la lectura dentro de Geopetro-Desktop.

2. Decisiones

Cada decisión presenta el problema que la motivó, lo que se decidió y lo que eso provoca en el sistema. Las referencias apuntan a las specs en specs/SDD.

D01 · Monorepo con aplicaciones independientes

Contexto	El sistema reúne recolección en campo, monitoreo web, histórico y simulación. Cada parte tiene un ritmo de entrega y una forma de despliegue diferentes.
Decisión	Seis aplicaciones desplegables en apps/, cada una con su propio build y Dockerfile. Braserv-Horus sigue separado y no será absorbido por Geopetro-Desktop.
Consecuencias	• Cada aplicación se construye y se despliega por sí sola. • La duplicación entre los dos desktops se acepta a propósito (DT-010). • Cuatro clases base se copian entre Core y Backend. Una biblioteca común solo se extraerá cuando un tercer servicio la necesite.
Referencias	README.md · software/system-overview.md · software/technical-debt.md

D02 · Braserv-Core es dueño de la identidad y del registro organizacional

Contexto	Usuario, empresa, regional, sector y unidad sirven a otros sistemas de Braserv, no solo a GeopetroIO.
Decisión	Esos registros y el inicio de sesión pertenecen a Braserv-Core. El Backend no copia tablas y lee mediante el contrato interno del Core. Cada servicio tiene su propia base de datos y su propio usuario en el mismo servidor MySQL: braserv_core y geopetro_io.
Consecuencias	• Un sistema nuevo reutiliza el inicio de sesión y la estructura sin depender del Backend. • En el Backend, unidad_id es una referencia lógica, sin llave foránea entre bases de datos. • Eliminar una unidad exige confirmación del Backend y de la Telemetría. Sin respuesta en 5 s, la eliminación se rechaza. Una unidad ya usada se inactiva, nunca se elimina.
Referencias	software/backend/braserv-core.md · RN-115 a RN-119

D03 · Solo el Core emite tokens, firmados con RS256

Contexto	Varios servicios necesitan confiar en la identidad del usuario sin compartir un secreto.
Decisión	El Core firma con la llave privada y publica las llaves públicas en el JWKS. El token de persona vale 1 h y el token de servicio vale 15 min. El campo tipo impide que un token de persona abra una ruta interna y que un token de servicio abra una ruta pública. El Backend consulta en el Core el acceso actual del usuario, con caché de 10 s.
Consecuencias	• Ningún consumidor guarda un secreto de firma. • Desactivar un usuario corta el acceso en máximo 10 s. • Si el Core se cae, el Backend usa el último acceso conocido hasta por 5 min y después lo niega. Es el costo aceptado para que un reinicio del Core no tumbe el tiempo real. • La revocación individual de tokens sigue pendiente (SEC-008).
Referencias	software/backend/braserv-core.md §6 · software/apis/braserv-core.md

D04 · Un solo origen a través del proxy

Contexto	El navegador habla con dos servicios, y las rutas internas no pueden quedar expuestas.
Decisión	El nginx del Front publica un único origen. Identidad y registros van al Core. Monitoreo, simulador y /ws van al Backend. Las rutas /internal/** y /.well-known/** no se publican. El Front nunca llama a la Telemetría.
Consecuencias	• No hay CORS entre servicios en el navegador. • La superficie pública es más pequeña. • El Backend valida el acceso a la unidad antes de consultar la Telemetría.
Referencias	software/system-overview.md §3

D05 · Dos caminos independientes para la telemetría

Contexto	El histórico no puede perder lecturas. El tiempo real solo necesita el estado más reciente.
Decisión	MQTT lleva el histórico hasta la Telemetría e InfluxDB. WebSocket con STOMP lleva el estado actual al Backend, que lo reenvía al Front sin guardarlo. En el Desktop, MQTT usa una cola de 3600 lecturas y el tiempo real guarda solo el último valor. Cada camino tiene su propio worker en un hilo virtual.
Consecuencias	• La falla de un camino no afecta al otro ni a la lectura del PLC. • La cola cubre cerca de una hora de broker caído. La base H2 local guarda el registro completo. • Al reconectarse, el tiempo real envía solo el estado más reciente. • Cada camino se puede apagar en el engranaje de configuración de la estación.
Referencias	software/apis/websocket-realtime.md · software/mqtt/mqtt-telemetria.md

D06 · MQTT por lotes con mensaje autodescriptivo

Contexto	Un mensaje por sensor multiplicaría el volumen, y los brokers administrados cobran por mensaje.
Decisión	Un mensaje por ciclo, QoS 1, en el tópico telemetria/{idUnidade}/batch. Cada lectura lleva tipo, serie y dirección, así que no existe un catálogo central. Solo el Desktop publica y solo la Telemetría consume. El Backend salió del broker.
Consecuencias	• El volumen es cerca de 5 veces menor que el formato por sensor. • Una lectura sin valor, como un peso aún sin calibrar, no se publica. • Pendientes: autenticación en el broker (SEC-009) y búfer de contingencia en el productor.
Referencias	software/mqtt/mqtt-telemetria.md

D07 · Servicio de Telemetría con InfluxDB

Contexto	La escritura es continua, las 24 horas, y las consultas piden horas de datos a 1 s.
Decisión	Un servicio propio guarda las series en InfluxDB en lotes asíncronos y siempre en UTC. Las consultas de más de 2000 puntos por serie se reducen. Un broker caído no impide que el servicio arranque.
Consecuencias	• La carga de escritura queda separada del MySQL transaccional. • La pantalla de histórico recibe un volumen controlado incluso en ventanas largas. • La Telemetría no conoce usuarios. Quien controla el acceso es el Backend.
Referencias	apps/geopetro-telemetria/specs · software/apis/rest-monitoramento.md

D08 · Cómo se identifica la unidad

Contexto	La web necesita un identificador estable. El contrato de recolección ya usaba el nombre de la unidad.
Decisión	El registro se llama Unidad en todo el sistema. Web, REST y WebSocket usan el id numérico. MQTT e InfluxDB usan el nombre de la unidad como llave. El Backend traduce uno en otro antes de consultar la Telemetría.
Consecuencias	• El Front trabaja con un único identificador. • Cambiar el nombre de una unidad parte la serie histórica en dos. Es el riesgo de mayor costo registrado (RN-018).
Referencias	RN-018 · RN-119 · software/backend/braserv-core.md §9

D09 · Desktop como agente de borde dirigido por configuración

Contexto	Cinco magnitudes fijas en el código impedían instrumentar equipos que no fueran taladros.
Decisión	Cada unidad tiene un documento de tarjetas con la conexión del PLC y la lista de tarjetas, cada una con tipo y dirección. Se edita solo en el Desktop, por ADMIN o SUPORTE, se guarda en el Backend con número de revisión, se entrega por el canal STOMP /cards y queda en caché en disco. Hay seis tipos cerrados. La lectura se hace en bloque, un rango del DB por ciclo.
Consecuencias	• Instrumentar un equipo nuevo pasa a ser un registro, no un proyecto. • Una unidad sin tarjetas no lee ni publica (RN-088). • Dos ediciones simultáneas se rechazan por la revisión, con error 409. • Al copiar de otra unidad, la IP del PLC no se copia.
Referencias	negocio/requisitos/cards-configuraveis.md · software/apis/configuracao-sonda.md

D10 · Identidad estable de la tarjeta

Contexto	El id de la tarjeta se vuelve tag en InfluxDB, con retención de 5 años. Un tag con valores libres degrada la base.
Decisión	El id se genera con el formato TIPO_NN, nunca cambia y no se reutiliza. El nombre es solo una etiqueta de pantalla. La tarjeta no se elimina, se desactiva. El límite es de 100 tarjetas por unidad.
Consecuencias	• La cantidad de tags en InfluxDB queda bajo control. • Toda serie antigua sigue teniendo una tarjeta que explica qué mide.
Referencias	RN-081 · RN-091 · software/mqtt/mqtt-telemetry.md §4

D11 · La calibración se queda en la estación

Contexto	La geometría del peso y de la llave hidráulica se miden en campo, en la propia unidad.
Decisión	La calibración de peso y torque queda en el computador de la unidad, en <code>config/calibracao-cards.json</code> , separada por el id de la tarjeta. El rango del transmisor queda en el documento de la unidad.
Consecuencias	• Los valores medidos en campo no viajan por la red. • Cambiar el computador exige rehacer o copiar la calibración.
Referencias	negocio/requisitos/cards-configuraveis.md §10b

D12 · El servidor registra la alarma y la estación la señala

Contexto	La supervisión y el cliente necesitan un histórico único. El equipo en la unidad necesita la alarma aun sin red.
Decisión	El Backend es el único productor del histórico de alarmas. Evalúa los límites remotos sobre el canal de tiempo real. La alarma es la única parte del sistema orientada a eventos: cada episodio es una secuencia de solo inserciones. La estación tiene sus propios límites locales, funciona sin red y sin inicio de sesión, y no genera eventos.
Consecuencias	• Existe una única fuente de verdad para el histórico de alarmas. • Una unidad con el tiempo real caído, o una tarjeta invisible, no alarma en el servidor aunque publique por MQTT. • La alarma local solo llega a la flota con la actualización automática del Desktop, que todavía no existe.
Referencias	negocio/requisitos/alarmes.md · negocio/requisitos/configuracao-da-estacao.md

D13 · Configuración restringida, con sesión solo en memoria

Contexto	Ajustar dirección, rack y slot exige estar en la unidad. Existe una credencial de servicio común a toda la flota.
Decisión	Configurar exige ADMIN o SUPORTE autenticado en línea. La sesión vive en memoria y no se guarda en disco. Sin iniciar sesión, el Desktop sigue leyendo, publicando y alarmando.
Consecuencias	• Ninguna credencial adicional queda guardada en el borde (SEC-011). • Configurar una estación exige internet.
Referencias	RN-086 · negocio/requisitos/configuracao-da-estacao.md §5

D14 · Despliegue separado por camino de datos

Contexto	La ingesta escribe sin parar y recibe conexiones de las unidades. La parte web es intermitente y guarda el registro de usuarios.
Decisión	Dos máquinas virtuales. La transaccional tiene Front, Core, Backend y MySQL. La de telemetría tiene Mosquitto, Telemetría e InfluxDB. También existe una alternativa en una sola máquina, con Caddy y TLS automático. Las bases MySQL se versionan con Flyway y se validan al arrancar.
Consecuencias	• Una falla en la web no interrumpe la ingesta. • Cada lado escala de forma independiente. • InfluxDB queda junto al consumidor, así la escritura continua no cruza la red. Entre las máquinas solo pasa la consulta REST.
Referencias	deploy/README.md · deploy/vm-unica/README.md

D15 · Arquitectura hexagonal por módulo

Contexto	Core y Backend tienen varios dominios y dependen uno del otro por red.
Decisión	Core y Backend son proyectos Maven con un módulo por dominio, organizados en puertos y adaptadores. La integración entre servicios pasa por puertos, como <code>AcessoDoUsuarioPort</code> y <code>CatalogoDeUnidadesPort</code> .
Consecuencias	• Cambiar un adaptador, como el cliente HTTP del Core, no toca la regla de negocio. • Las pruebas de dominio corren sin red.
Referencias	software/backend/braserv-core.md §7 y §10

3. Riesgos aceptados y pendientes

Estos puntos son conocidos y fueron aceptados o están esperando una decisión.

Punto	Efecto	Situación
Broker STOMP en memoria	Con dos instancias del Backend, el tiempo real no pasa de una a la otra.	Aceptado mientras haya una sola instancia. Solución prevista: broker externo.
Broker MQTT sin autenticación	Cualquier cliente en la red puede publicar en el tópico de telemetría.	Pendiente (SEC-009).
Nombre de la unidad como llave	Cambiar el nombre parte la serie histórica.	Riesgo aceptado (RN-018).
Sin actualización automática del Desktop	Las correcciones solo llegan a las unidades con una visita.	Pendiente.
Búfer de contingencia MQTT	Un broker caído por más de una hora descarta las lecturas más antiguas de la cola.	Pendiente (P-07). El H2 local conserva el registro.
Token sin revocación individual	Un token filtrado vale hasta que vence.	Pendiente (SEC-008).
Repositorios dentro de OneDrive	La sincronización ya dañó un repositorio Git.	Pendiente (DT-004).

Dónde profundizar

Tema	Documento
Visión del sistema	specs/SDD/software/system-overview.md
Braserv-Core	specs/SDD/software/backend/braserv-core.md
Tiempo real	specs/SDD/software/apis/websocket-realtime.md
Telemetría MQTT	specs/SDD/software/mqtt/mqtt-telemetry.md
Tarjetas configurables	specs/SDD/negocio/requisitos/cards-configuraveis.md
Alarmas	specs/SDD/negocio/requisitos/alarmes.md
Despliegue	deploy/README.md
Reglas y pendientes	specs/SDD/negocio/reglas/business-rules.md · requisitos/open-questions.md